TwinMapper

Architecture Document

Compile-time-first, schema-driven code generation and mapping platform

Version 1.0 March 2026

Table of Contents

Table of Contents ... 2
Overview ... 3
Architectural Principles ... 4
Two-Engine Architecture ... 5
  Document Engine ... 5
  Object Engine ... 5
  High-Level Architecture Diagram ... 5
Module Layer Map ... 6
  Module Families ... 6
Dependency Spine ... 7
Document Engine Architecture ... 8
  Build-Time Flow ... 8
  Runtime Flow ... 9
    NodeCursor Contract ... 9
Object Engine Architecture ... 10
  Build-Time Flow ... 10
  Runtime Flow ... 10
  Mapper Types ... 10
  Supported Mapping Flows ... 10
    Proxy Safety ... 11
Internal Meta-Model ... 12
Spring Integration Architecture ... 13
  Auto-Configuration Level ... 13
  Conversion Level ... 13
  Validation Level ... 13
  Spring Integration Points ... 13
Security Model ... 14
Hard Non-Goals ... 15
Technology Stack ... 15

Overview

TwinMapper is a compile-time-first, schema-driven code generation and mapping platform for Java. It generates strongly typed DTOs, enums, binders, validators, metadata descriptors, and typed object mappers from fixed YAML, JSON, and BPMN definitions. It provides runtime document binding and typed layer-to-layer object mapping without reflection-heavy inference or heuristic mapping as the core behavior.

The platform is built on Spring ecosystem defaults and integrates naturally with Spring Boot, Spring Validation, Spring Conversion, and Spring configuration patterns. Rather than assembling multiple external tools, TwinMapper provides a unified Spring-first platform for definition-driven binding, object mapping, validation, and code generation.

Central Architectural Rule

All primary behavior is generated at build time from fixed definitions. Runtime behavior executes the generated code. Nothing is inferred at runtime from live payloads.

Architectural Principles

Compile-time first — DTOs, binders, validators, and mappers are generated at build time. No runtime inference.

Definition first — The internal meta-model is the single source of truth. All generated artifacts derive from it.

Strongly typed — No reflection-heavy runtime inference. No heuristic field guessing.

Deterministic — Identical inputs always produce identical outputs across builds.

Separation of concerns — Document binding and object mapping are independent engines sharing a common foundation.

Spring-native — Integration with Spring validation, conversion, resource-loading, and auto-configuration is first-class.

Generic core — No domain-specific semantics in the core platform. Domain knowledge lives in consuming applications.

Two-Engine Architecture

TwinMapper's architecture is split into two primary engines — the Document Engine and the Object Engine — supported by a shared foundation, three format modules, build tooling, and a Spring Boot integration layer.

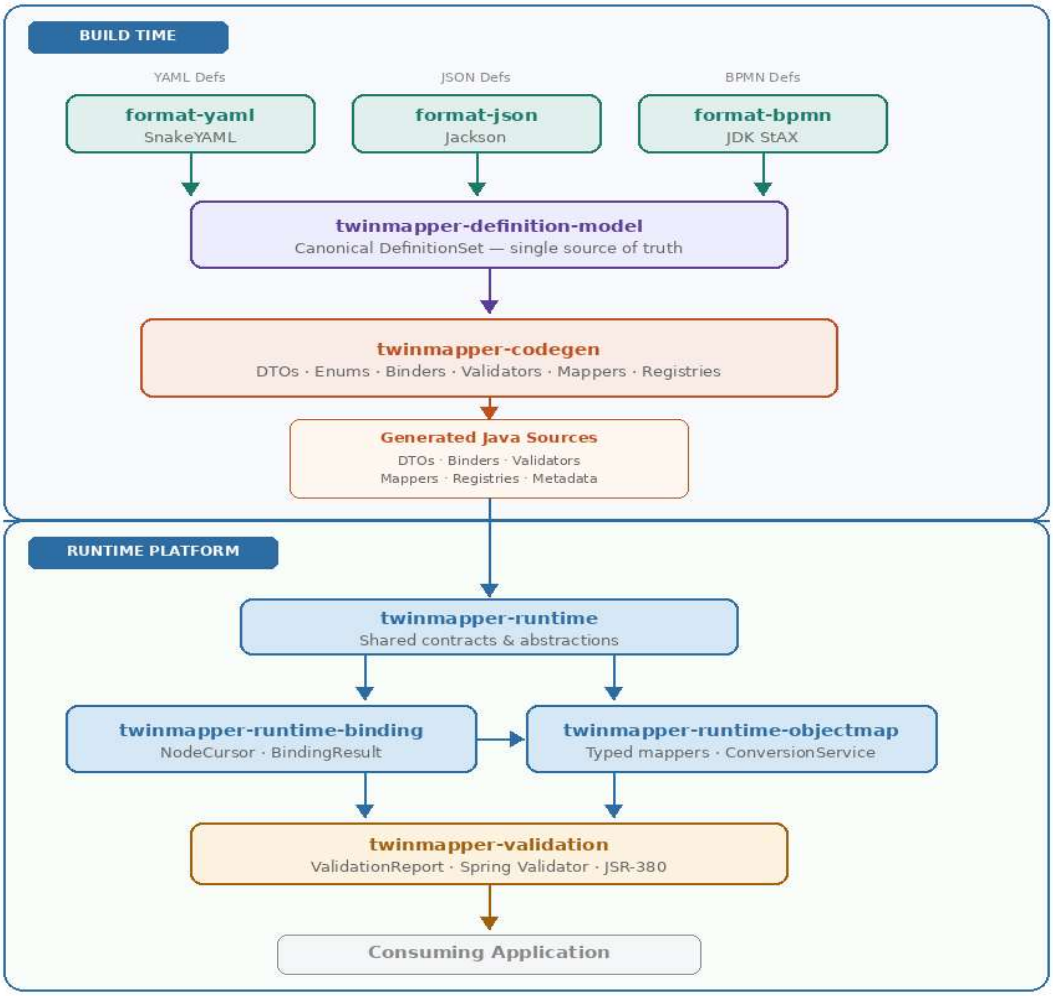
Document Engine

Reads fixed YAML, JSON, and BPMN definitions at build time. Generates DTOs, enums, binders, validators, registries, and metadata. At runtime, binds actual YAML, JSON, and BPMN XML documents into generated DTOs using a NodeCursor abstraction. No reflection at binding time.

Object Engine

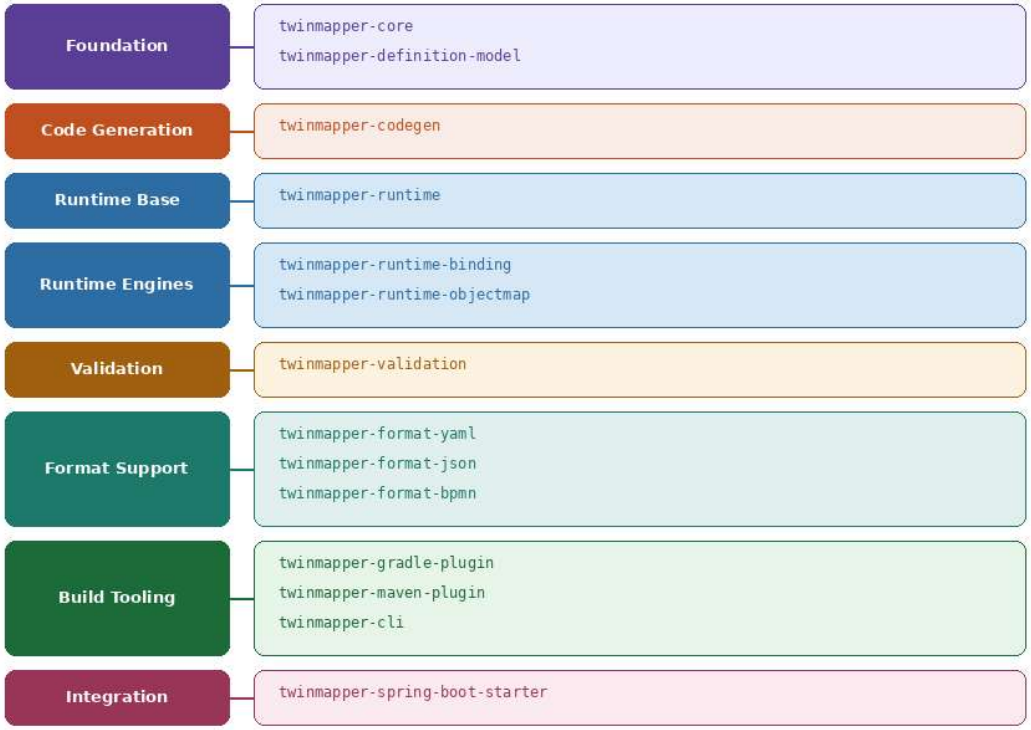
Generates typed Java-to-Java mappers for layer-crossing flows. Supports entity-to-domain, domain-to-DTO, DTO-to-entity, entity-to-projection, request/command-to-domain, and domain-to-event/view/response. Three mapper modes: CREATE, UPDATE, and PATCH.

High-Level Architecture Diagram



Module Layer Map

TwinMapper contains 14 core modules organized into five families. The layer map shows the vertical dependency order from foundation to integration.

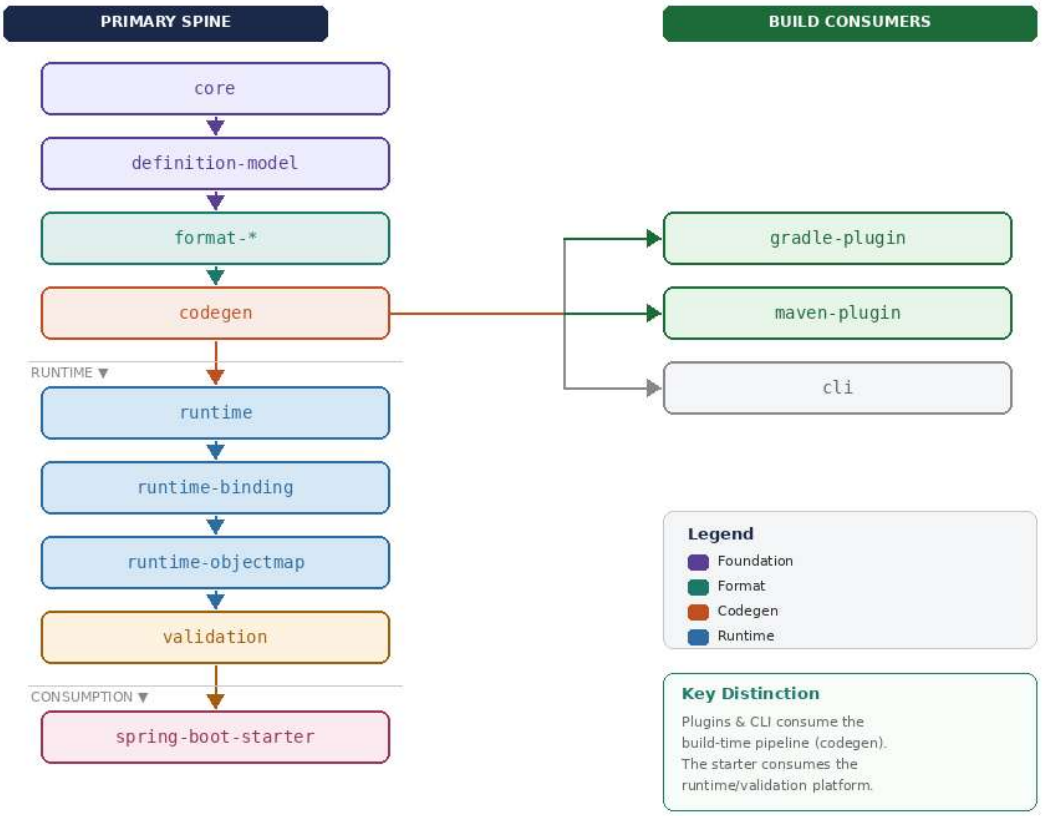


Module Families

Family	Modules	Purpose
Core	core, definition-model, codegen, validation	Foundation, meta-model, generation, validation
Runtime	runtime, runtime-binding, runtime-objectmap	Shared contracts, document binding, object mapping
Format	format-json, format-yaml, format-bpmn	Definition readers and runtime parsers
Tooling	gradle-plugin, maven-plugin, cli	Build integration and developer tooling
Integration	spring-boot-starter	Spring Boot auto-configuration

Dependency Spine

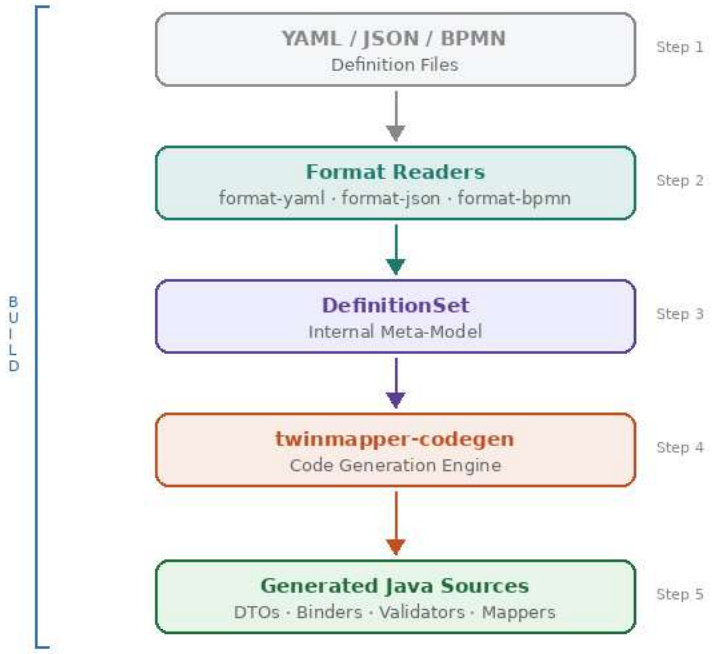
The locked dependency spine defines the linear chain from foundation through to application integration. All modules depend downward only — no circular dependencies are permitted.



Document Engine Architecture

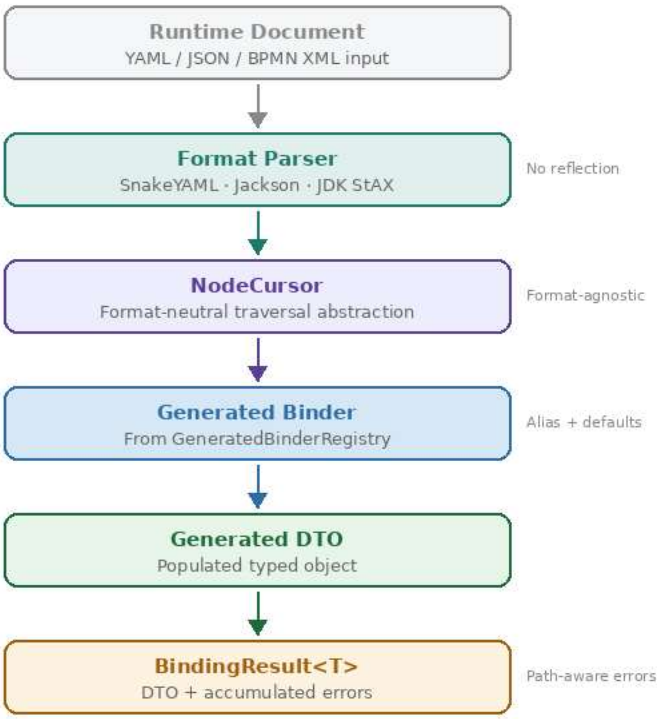
Build-Time Flow

The build plugin scans configured definition directories for YAML, JSON, and BPMN definition files. The appropriate format reader parses each file into the internal DefinitionSet model. The codegen engine consumes the DefinitionSet and generates Java sources. Generated sources are written to build/generated/sources/twinmapper and compile together with handwritten code.



Runtime Flow

At runtime, the consuming application provides a YAML, JSON, or BPMN XML document as input. The format parser produces a raw parsed structure which is adapted into a NodeCursor — the format-neutral traversal abstraction. The generated binder reads fields, applies aliases, defaults, and required-field rules, then constructs the generated DTO.



NodeCursor Contract

NodeCursor is the core runtime abstraction that decouples format parsing from binding logic. Each format module provides its own NodeCursor implementation. Generated binders only operate on NodeCursor — they have no direct dependency on Jackson, SnakeYAML, or StAX. This is what makes TwinMapper format-agnostic at the binding layer.

Object Engine Architecture

Build-Time Flow

The build plugin scans the configured mapping definition directory for YAML mapping DSL files. The YAML format reader parses mapping files into ObjectMappingDefinition entries in the meta-model. The codegen engine generates typed mapper classes for each declared mapping.

Runtime Flow

The consuming application calls TwinMapperRuntime.map(source, TargetType.class). The TwinObjectMapperRegistry resolves the correct generated mapper by source and target type. The mapper executes field assignments, converter calls, and nested mapper delegations as plain Java method calls. For UPDATE mode, BeanWrapper handles nested property path resolution and JPA proxy transparency. For PATCH mode, null fields in the source are skipped.

Mapper Types

Mapper	Mode	Null Policy	Behavior
TwinObjectMapper<S,T>	CREATE	SET_NULLS	Builds new target from source
TwinUpdateMapper<S,T>	UPDATE	IGNORE_NULLS	Applies source onto existing target
TwinPatchMapper<S,T>	PATCH	IGNORE_NULLS (fixed)	Applies only non-null source fields
TwinInverseMapper	Reverse	Per original	Reverse mapping via inverseOf declaration

Supported Mapping Flows

Source	Target	Typical Use Case
Entity	Domain	Load aggregate from persistence
Domain	Entity	Persist aggregate
Domain	DTO	API response serialization
DTO	Domain	Command construction from request

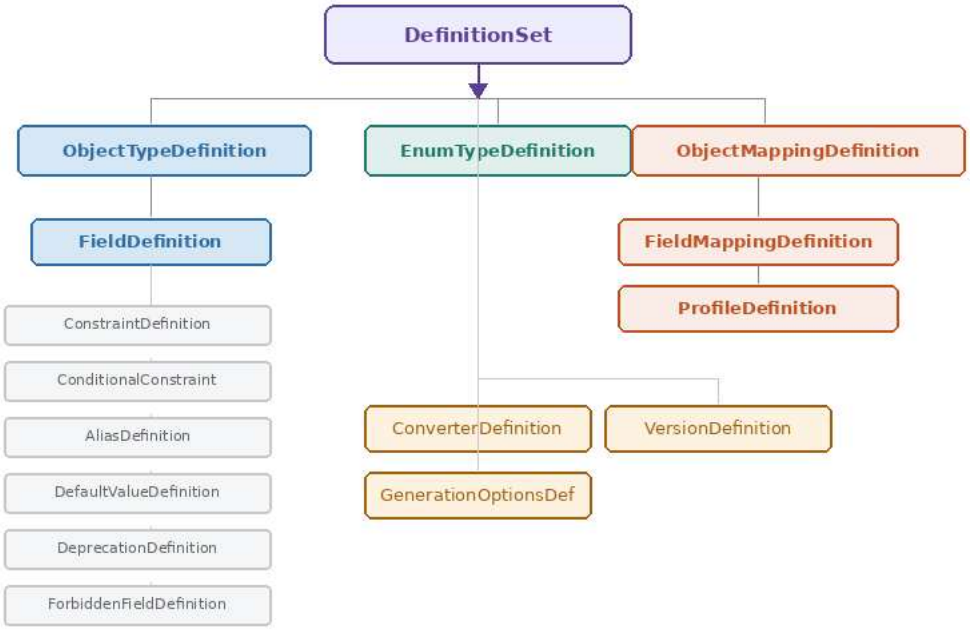
Entity	DTO	Read-model projection shortcut
DTO	Entity	Direct persistence write from API
Entity	Projection	Read-model construction
Request/Command	Domain	Domain command from HTTP request
Domain	Response/View/Event	CQRS projection, event payload
Patch DTO	Entity	Partial update to persistence entity
Patch DTO	Domain	Partial update to domain aggregate

Proxy Safety

All reflection-based operations in the object engine use AopUtils.getTargetClass(), AopProxyUtils.ultimateTargetClass(), and ProxyUtils.getUserClass() to unwrap Spring-managed proxies before inspecting types. This is mandatory — not optional. Omitting these causes silent failures on proxied targets.

Internal Meta-Model

All format readers normalize their inputs into one canonical internal model. This model is the single source of truth for all code generation. It is format-neutral and independent of any domain language.



Spring Integration Architecture

TwinMapper integrates with Spring at three levels: auto-configuration, conversion, and validation.

Auto-Configuration Level

The spring-boot-starter provides @AutoConfiguration with AutoConfiguration.imports. All TwinMapper beans are conditionally auto-configured. TwinMapperProperties via @ConfigurationProperties(prefix="twinmapper") is the central configuration contract.

Conversion Level

The object engine's converter registry integrates with Spring's ConversionService. Converters implementing Converter, ConverterFactory, or GenericConverter registered in the Spring application context are available to TwinMapper without additional registration.

Validation Level

Generated validators implement Spring's Validator interface. ValidationReport errors translate cleanly into Spring MVC exception types. JSR-380 annotations work alongside generated validators.

Spring Integration Points

Integration Area	Spring Components Used
Validation	Validator, SmartValidator, Errors, BindingResult, @Valid, @Validated, JSR-380
Conversion	ConversionService, Converter, ConverterFactory, GenericConverter
Property Access	BeanWrapper, DirectFieldAccessor
Proxy Safety	AopUtils, AopProxyUtils, ProxyUtils
Resource Loading	ResourceLoader, PathMatchingResourcePatternResolver
Configuration	@ConfigurationProperties, @AutoConfiguration, AutoConfiguration.imports
Utilities	Assert, StringUtils, ObjectUtils, ClassUtils, AnnotationUtils, ReflectionUtils
SPI Ordering	OrderComparator, PriorityOrdered, @Order, Ordered

Security Model

TwinMapper enforces security constraints at the parsing layer. These are non-negotiable — omitting any of them is a security vulnerability.

BPMN — XXE Protection

XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES = false and XMLInputFactory.SUPPORT_DTD = false. Both required. Omitting either allows XML External Entity attacks.

YAML — Safe Construction

new Yaml(new SafeConstructor(new LoaderOptions())) is mandatory. The default SnakeYAML constructor allows arbitrary Java object deserialization and is prohibited.

Resource Loading

All classpath file resolution uses Spring ResourceLoader. Direct ClassLoader.getResourceAsStream() calls are prohibited.

Hard Non-Goals

The following are explicitly excluded from all TwinMapper implementation.

Non-Goal	Reason
Uncontrolled live schema learning	Contradicts compile-time-first principle
Silent hidden guessing	All mapping must be explicit and auditable
Runtime-first architecture	Compile-time-first is locked
Domain-specific semantics in core	Core platform remains generic

Technology Stack

Dependency	Version	Purpose
spring-core	6.x+	Utilities, ResourceLoader, ConversionService SPI
spring-context	6.x+	Auto-configuration contracts
spring-boot	3.x+	@ConfigurationProperties, AutoConfiguration
jackson-databind	2.14+	JSON format support
snakeyaml	2.x	YAML format support (bundled in Spring Boot)
JDK StAX	Built-in	BPMN XML parsing — no vendor dependency
Java	17+	Records, sealed interfaces, modern features

No mandatory runtime dependency on:

Any BPMN library, Lombok, MapStruct, Micronaut, or Hibernate Validator (optional; provided only if JSR-380 validation is needed).